Intrinsic Value Analyzer

Quantitative Stock Market Asset Analysis Model

Model Card V2.3

2026-05-17

Nilrudra Mukhopadhyay

Document Purpose

The following document serves as a report on the performance of the Intrinsic Value Analyzer model, and as a guide to its structure, intended use, and possible future improvements. Sections of this report include basic model information, training data structure, evaluation process, results, known limitations, and possible improvements.

Model Information

The intrinsic value analyzer is built on the idea of estimating a stock's "real" price based on a formula first developed and introduced by Benjamin Graham. The formula uses two constants, the P/E ratio of a stock with no growth at 8.5, and the average yield of a AAA corporate bond at 4.4. It then expresses the Intrinsic Value as a dollar amount, which is the "market corrected" share price.

$$V = \frac{EPS \times (8.5 + 2g) \times 4.4}{Y}$$

Figure 1: Benjamin Graham's Intrinsic Value formula

The issue with Benjamin Graham's formula is that the model breaks down when faced with non-standard or cyclical businesses where the cash flow is irregular and the P/E changes with seasonal business downturns. The formula inherently requires stocks with continuous stable income and "ideal" circumstances, and while trained analysts can recognize value traps when using this formula, it isn't usable for average investors. However, Machine Learning optimization can be used to potentially bridge that skill gap through mathematically built "intuition".

Key Model Dependencies

1. Joblib - for loading the serialized model artifact file
2. Pandas - when handling data matrices and array transformations
3. Scikit-learn (maybe) - loading MLPClassifier and StandardScaler

Model Inputs

The model needs a Pandas DataFrame row containing [P/E ratio, Intrinsic Value, Price] all formatted as decimal values (floats).

1. P/E ratio - provide the current P/E (accepts $0 values or negative values)
2. Intrinsic Value - calculate using Benjamin Graham's formula before feeding the model
3. Price - input the current price or end-of-day trailing share price

Crucial Operational Note: when implementing the model any raw inputs must pass through the loaded StandardScaler first via transform() before being handed to the Neural Network.

Model Outputs

Different Neural Network model methods will produce different results. The predict_proba(x_scaled) method returns the raw probability distribution for both classes (1 and 0) which could be useful for showing model confidence. Meanwhile, the predict(x_scaled) function outputs a binary integer value.

- 1 represents a stock which the model predicts as “Undervalued” forecasting the price will rise
- 0 represents a stock which the model predicts as “Overvalued” forecasting the price going down

Model Architecture

The architecture during training is broken into two layers, layer 1 computes the Intrinsic Value as an engineered feature, while layer 2 performs value standardization and model training/evaluation.

1. Layer 1 Feature Engineering: calculates the "Graham” Intrinsic value of the stock.
    - Inputs: price, EPS, P/E, expected growth, and AAA corporate bond Yield
    - Outputs: one primary feature the IV expressed as a dollar amount
2. Layer 2 Supervised Learning Model: A Neural Network (Logistic Regressive) trained on Historical data which compares P/E ratios, Graham Intrinsic Values, and the current price against the real outcomes of price movement between 2021–2025.
    - Training Label 1 = Winner: the stock grew over the past 4 years
    - Training Label 0 = Loser: the stock fell over the past 4 years
    - Action: the NN learns to "veto" IV results when it sees a historical pattern of "Value Traps" in relation to the P/E ratios.

The model during production consists of a package of artifacts which contain the model, and the model’s StandardScaler. For the most effective forecast, the scaler must be used before feeding the model any new asset metrics, which ensures the model makes predictions on values scaled similarly to its training data.

Training Dataset Sources

Kaggle was the source of all publicly available open datasets used during the development of this model, while the yFinance API (free tier) provided historical and current stock prices, from 2017 to 2021. The yFinance API data was used for calculating compounded annual growth rates and labeling stock growths as “overvalued” or “undervalued”. The kaggle datasets alternatively, were used to extract stock metrics such as names, tickers, prices, P/E, EPS values, and etc. Below are the links to all Kaggle datasets used during the data acquisition process.

US stock index comprehensive data
s&p 500 EPS and PE ratio
Current Corporate AAA bond yield value
s&p 500 growth rate price/returns

Training Data Structure

The data is housed within a local Microsoft SQL Express Database. Interfacing with this database was done through the use of several T-SQL scripts written/executed through SSMS, and python 3 scripts written/executed within a virtual environment using SQLalchemy. Below is the structured SQL used to create the tables for the training data.

```sql
CREATE TABLE dbo.stocks (
    ticker      VARCHAR(10)   PRIMARY KEY,
    stock_name  VARCHAR(255)  NOT NULL,
    sector      VARCHAR(255)  NOT NULL,
    price       DECIMAL(18,2) NOT NULL,
    earnings    DECIMAL(18,2) NOT NULL,
    pe_ratio    DECIMAL(18,2) NOT NULL,
    market_cap  DECIMAL(20,2) NOT NULL
);

CREATE TABLE dbo.growth_rates (
    ticker          VARCHAR(10),
    start_value     DECIMAL(18, 2) NOT NULL,
    end_value       DECIMAL(18, 2) NOT NULL,
    growth_rate     DECIMAL(18, 2) NOT NULL,
    corp_bond_yield DECIMAL(18, 2) NOT NULL
    CONSTRAINT      fk_stock_growth_rates
        FOREIGN KEY (ticker)
        REFERENCES  dbo.stocks(ticker)
);

CREATE TABLE dbo.historic_valuations (
    ticker          VARCHAR(10),
    valuation       VARCHAR(12),
    CONSTRAINT      fk_historic_valuations
        FOREIGN KEY (ticker)
        REFERENCES  dbo.stocks(ticker)
);
```

Figure 2: Structure T-SQL used to create the tables for model training data

During model training, pulling data into a Jupyter Notebook was done through a combination of the SQL query shown below and a Pandas dataframe.

```python
query = """
SELECT
    s.ticker, s.price, s.earnings, s.pe_ratio,
    g.growth_rate, g.corp_bond_yield, h.valuation
FROM dbo.stocks s
JOIN dbo.growth_rates g ON s.ticker = g.ticker
JOIN dbo.historic_valuations h ON s.ticker = h.ticker
WHERE s.price > 0
"""
```

Figure 3: Query string from the model training code